

CS407 Lecture 2

Chapter 2 (Part 1): A Simple
Syntax- Directed Translator

A Simple Syntax-Directed Translator

- In this chapter, the emphasis is on the **front end** of a compiler, in particular on **lexical analysis**, **parsing**, and **intermediate code generation**.

A Simple Syntax- Directed Translator

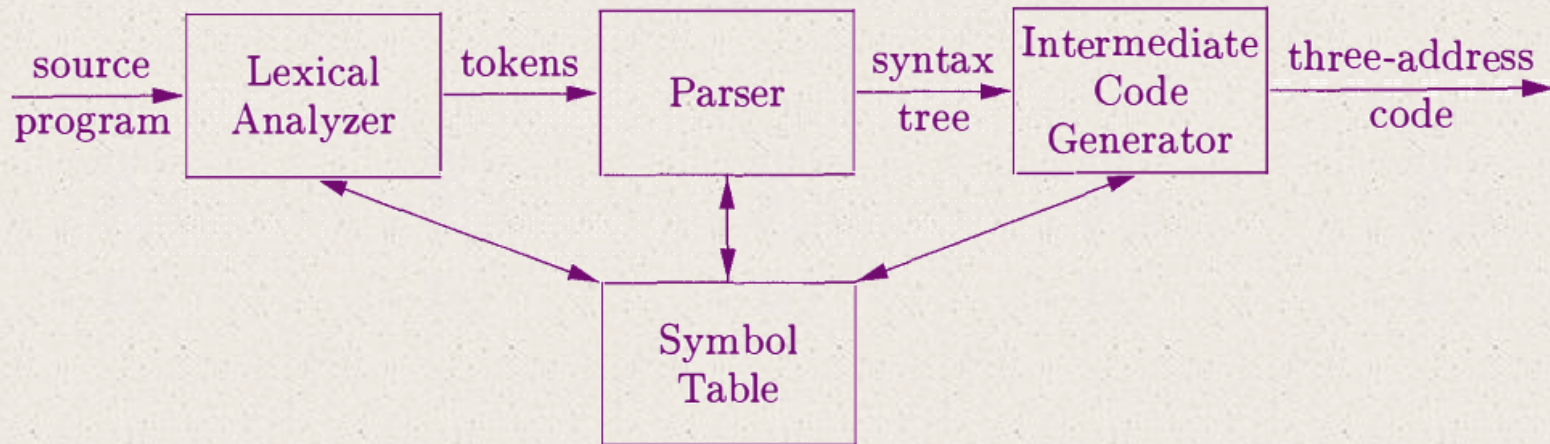


Figure 2.3: A model of a compiler front end

A Simple Syntax-Directed Translator

- The **analysis phase** of a compiler breaks up a source program into constituent pieces and produces an internal representation for it, called **intermediate code**.
- The **synthesis phase** translates the intermediate code into the **target program**.

A Simple Syntax-Directed Translator

- The **syntax** of a programming language describes **the proper form of its programs**.
- For specifying syntax, we present a widely used notation, called **context-free grammars** or **BNF** (for Backus-Naur Form).

A Simple Syntax-Directed Translator

- The **semantics** of the language defines what its programs mean; that is, what each program does when it executes.
- With the notations currently available, the semantics of a language is much more difficult to describe than the syntax.
- A **context-free grammar** can be used to help guide the translation of programs.

A Simple Syntax- Directed Translator

- We introduce a grammar-oriented compiling technique known as syntax-directed translation.
- We consider the syntax-directed translation of infix expressions to postfix form.
- The simple translator handles expressions like $9 - 5 + 2$, consisting of digits separated by plus and minus signs.

Syntax Definition: Context-Free grammar

- We introduce a notation - the "**context-free grammar**," or "**grammar**" for short - that is used to **specify the syntax of a language**.
- **For example**, an **if-else** statement in **Java** can have the form

if (expression) statement **else** statement

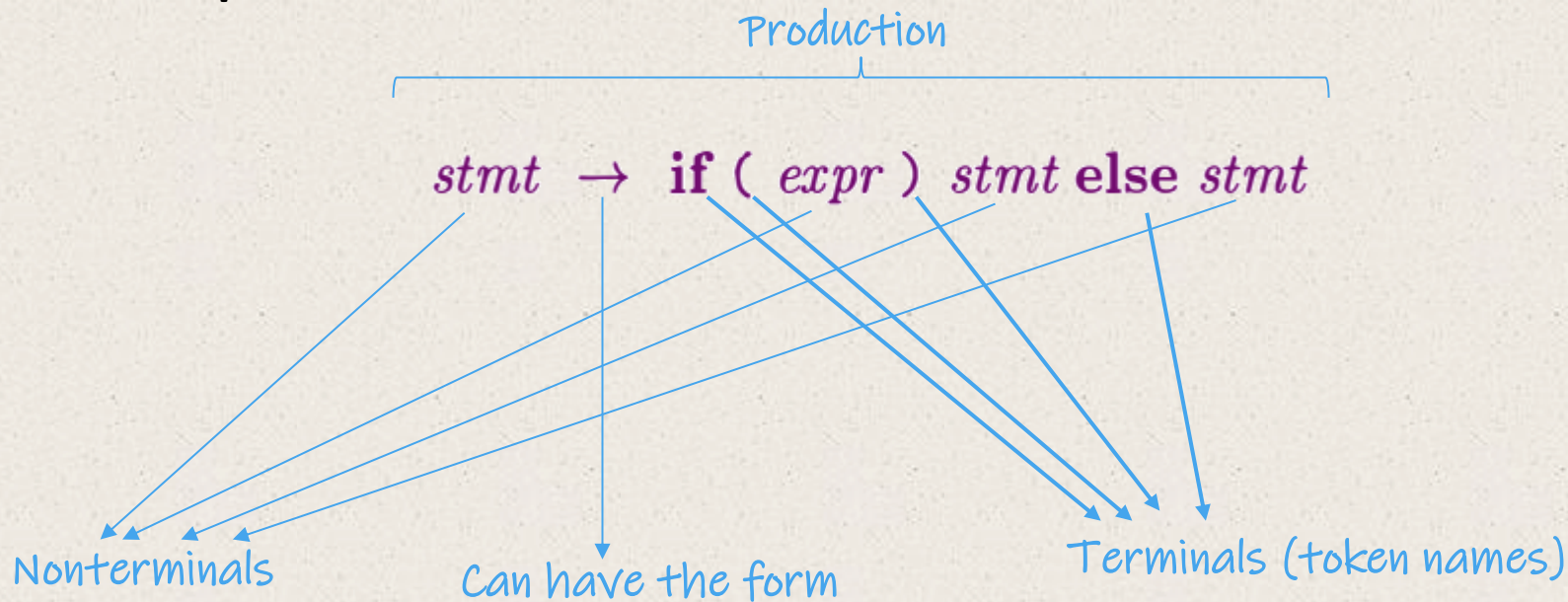
Syntax Definition: Context-Free grammar

if (expression) statement **else** statement

- An **if-else** statement is the concatenation of:
 - the keyword **if**,
 - an **opening parenthesis**,
 - an **expression**,
 - a **closing parenthesis**,
 - a **statement**,
 - the keyword **else**,
 - and another **statement**.

Syntax Definition: Context-Free grammar

- Using the variable **expr** to denote an **expression** and the variable **stmt** to denote a **statement**, this structuring **rule** can be expressed as



We refer to **tokens** and **terminals** synonymously.

Definition of Grammars

- A context-free grammar has four components:
 - 1) A set of terminal symbols, sometimes referred to as "tokens." The terminals are the elementary symbols of the language defined by the grammar.
 - 2) A set of nonterminals, sometimes called "syntactic variables." Each nonterminal represents a set of strings of terminals.

Definition of Grammars

- 3) A set of **productions**, where each production consists of **a nonterminal**, called the **head** or **left side** of the production, an **arrow**, and **a sequence of terminals and/or nonterminals**, called the **body** or **right side** of the production.
- 4) A designation of one of the **nonterminals** as **the start symbol**.

Definition of Grammars

- We specify grammars by listing their productions, with the productions for the start symbol listed first.
- We assume that digits, signs such as $<$ and $<=$, and boldface strings such as **while** are terminals.
- An italicized name is a nonterminal.
- Any nonitalicized name or symbol may be assumed to be a terminal.
- Productions with the same nonterminal as the head can have their bodies grouped, with the alternative bodies separated by the symbol $|$, which we read as "or".

Definition of Grammars: Example 2.1

- We consider the grammar for **lists of digits separated by plus or minus signs**.
 - e.g., 9-5+2, 3-1 , or 7.
- The following grammar describes the syntax of these expressions:

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \quad (2.1)$$

$$\textit{list} \rightarrow \textit{list} - \textit{digit} \quad (2.2)$$

$$\textit{list} \rightarrow \textit{digit} \quad (2.3)$$

$$\textit{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \quad (2.4)$$

Definition of Grammars: Example 2.1

- The bodies of the **three** productions with nonterminal **list as head** equivalently can be **grouped**:

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \mid \textit{list} - \textit{digit} \mid \textit{digit}$$

- The **terminals** of the grammar are the symbols **+ - 0 1 2 3 4 5 6 7 8 9**
- The **nonterminals** are the **italicized** names **list** and **digit**, with **list** being the **start symbol** because its productions are given first.

Definition of Grammars: Example 2.1

- A string of terminals is a sequence of zero or more terminals.
- The string of zero terminals, written as ϵ , is called the empty string.

Definition of Grammars: Summary

Anatomy of a Context-Free Grammar

1. Terminals (Tokens)

The elementary symbols of the language. (e.g., while, +, digits)

2. Nonterminals (Variables)

Representatives for sets of strings. They act as placeholders for structure.

3. Productions (Rules)

The mapping of structure.

Format: Head $\rightarrow$ Body

Example: $A \rightarrow XYZ$

4. Start Symbol

The designated nonterminal where the derivation begins.

Definition of Grammars: Summary

Grammar Notations & Conventions

1. Terminals are Bold or Signs

Example: **while**, **if**, **+**, **-**, **<=**

2. Nonterminals are *Italicized*

Example: *expr*, *stmt*, *list*

3. The Pipe | means 'OR'

Used to separate alternative bodies for the same head.

`list` $\rightarrow$ `list + digit` | `list - digit`

4. Epsilon (ϵ)

Represents the empty string (a string of zero terminals).

Derivations

- A grammar **derives** strings by beginning with **the start symbol** and **repeatedly** replacing a nonterminal by the body of a production for that nonterminal.
- The terminal strings that can be derived from the start symbol form the **language** defined by the grammar.

Derivations: Example 2.2

- The language defined by the grammar of Example 2.1 consists of lists of digits separated by plus and minus signs.

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \quad (2.1)$$

$$\textit{list} \rightarrow \textit{list} - \textit{digit} \quad (2.2)$$

$$\textit{list} \rightarrow \textit{digit} \quad (2.3)$$

$$\textit{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \quad (2.4)$$

Derivations: Example 2.2

Proving that $9 - 5 + 2$ is a list

Step 1: 9 is a list.

Rule: $\text{list} \rightarrow \text{digit}$



Step 2: $9 - 5$ is a list.

Rule: $\text{list} \rightarrow \text{list} - \text{digit}$.

Logic: 9 is a list, 5 is a digit.



Step 3: $9 - 5 + 2$ is a list.

Rule: $\text{list} \rightarrow \text{list} + \text{digit}$.

Logic: $9-5$ is a list, 2 is a digit.

Derivation: Repeatedly replacing a nonterminal with the body of a production.

Derivations: Example 2.3

- We start to develop a grammar for a function call.
 - E.g. `max (x , y)` of function `max` with parameters `x` and `y`.
 - E.g. `max ( )` with an empty list of parameters.

$$\begin{aligned} call &\rightarrow id (optparams) \\ optparams &\rightarrow params \mid \epsilon \\ params &\rightarrow params , param \mid param \end{aligned}$$

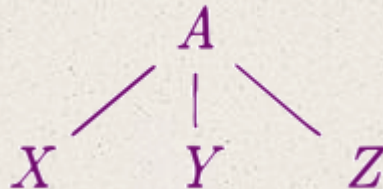
- We have not shown the productions for `param`, since parameters are really arbitrary expressions.

Parsing

- **Parsing** is the problem of taking a string of terminals and figuring out **how to derive it from the start symbol of the grammar**.
- If it **cannot be derived** from the start symbol of the grammar, then **reporting syntax errors** within the string.

Parse Trees

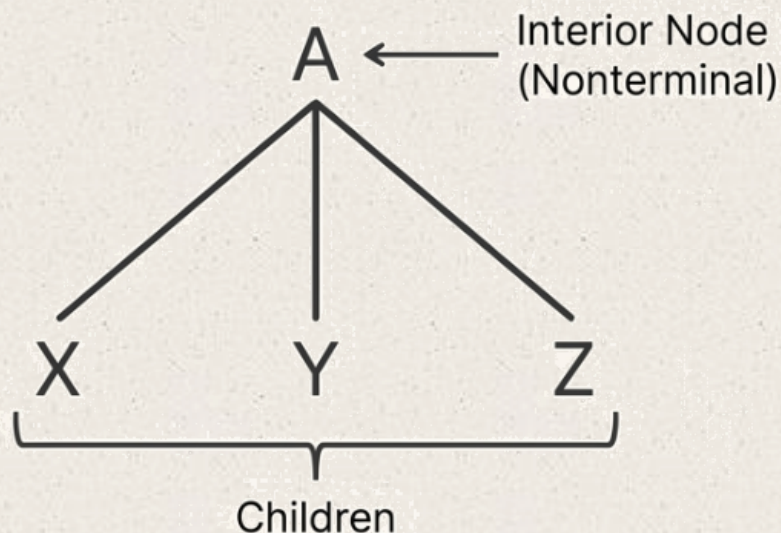
- A parse tree pictorially shows how the start symbol of a grammar derives a string in the language.
- If nonterminal A has a production $A \rightarrow XYZ$, then a parse tree may have an interior node labeled A with three children labeled X , Y , and Z , from left to right:



Parse Trees

- Formally, given a context-free grammar, a parse tree according to the grammar is a tree with the following properties:
 - 1) The root is labeled by the start symbol.
 - 2) Each leaf is labeled by a terminal or by ϵ .
 - 3) Each interior node is labeled by a nonterminal.
 - 4) If A is the nonterminal labeling some interior node and $X_1, X_2, \dots, X_n$ are the labels of the children of that node from left to right, then there must be a production $A \rightarrow X_1 X_2 \dots X_n$.
 - Here, $X_1, X_2, \dots, X_n$ each stand for a symbol that is either a terminal or a nonterminal.
 - As a special case, if $A \rightarrow \epsilon$ is a production, then a node labeled A may have a single child labeled ϵ .

Parse Trees



Geometric Rules

1. **Root:** Labeled by the start symbol.
2. **Interior Nodes:** Labeled by nonterminals.
3. **Leaves:** Labeled by terminals or ϵ .
4. **Yield:** Reading leaves left-to-right forms the generated string.
5. **Logic:** If Node A has children X, Y, Z, there implies a production $A \rightarrow XYZ$.

Parse Trees: Example 2.4

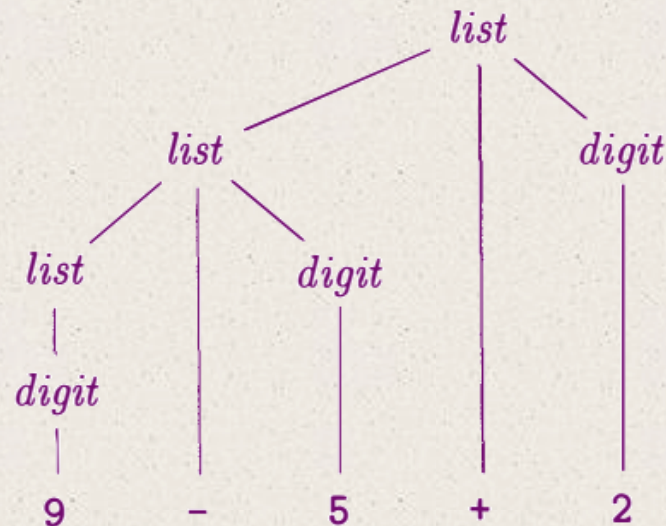
- The derivation of $9-5+2$ in Example 2.2 is illustrated by the tree in Fig. 2.5.

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \quad (2.1)$$

$$\textit{list} \rightarrow \textit{list} - \textit{digit} \quad (2.2)$$

$$\textit{list} \rightarrow \textit{digit} \quad (2.3)$$

$$\textit{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \quad (2.4)$$



Parse Trees: Example 2.4

- From left to right, the leaves of a parse tree form the yield of the tree, which is the string generated or derived from the nonterminal at the root.
- In Fig. 2.5, the yield is $9-5+2$.

Parse Trees

- Another definition of the language generated by a grammar is as the set of strings that can be generated by some parse tree.
- The process of finding a parse tree for a given string of terminals is called parsing that string.

Ambiguity

- A grammar can have more than one parse tree generating a given string of terminals.
- Such a grammar is said to be ambiguous.
- To show that a grammar is ambiguous, all we need to do is find a terminal string that is the yield of more than one parse tree.
- A string with more than one parse tree usually has more than one meaning.
- We need to design unambiguous grammars.

Ambiguity: Example 2.5

- Suppose we used a **single nonterminal string** and did not distinguish between digits and lists, as in **Example 2.1**.
- We could have written the grammar

$string \rightarrow string + string \mid string - string \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

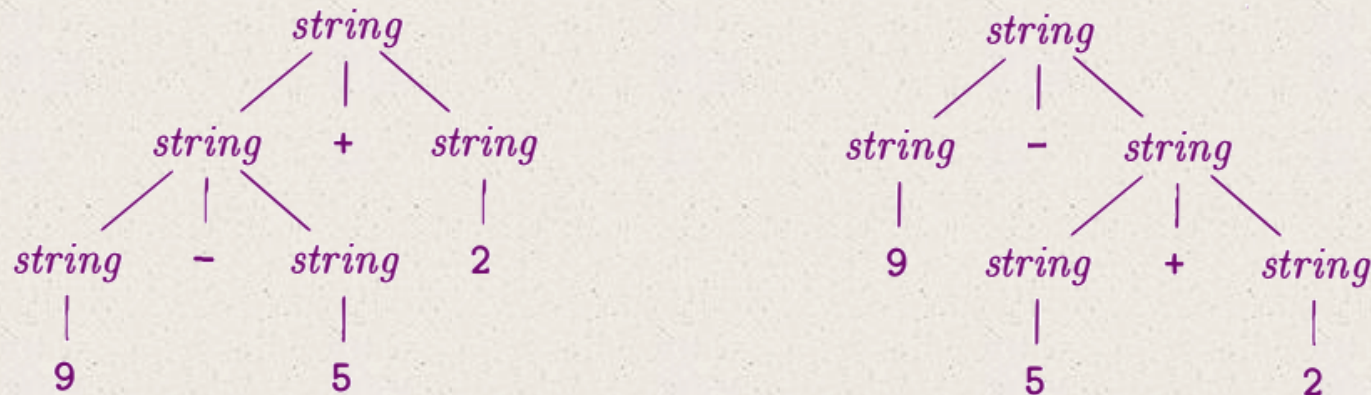


Figure 2.6: Two parse trees for 9-5+2

Ambiguity: Example 2.5

- Fig. 2.6 shows that an expression like $9-5+2$ has more than one parse tree with this grammar.
- The two trees for $9-5+2$ correspond to the two ways of parenthesizing the expression:
 - $(9-5)+2$ and $9-(5+2)$.
- This second parenthesization gives the expression the unexpected value 2 rather than the customary value 6.
- The grammar of Example 2.1 does not permit this interpretation.

Associativity of Operators

- By convention, $9+5+2$ is equivalent to $(9+5)+2$ and $9-5-2$ is equivalent to $(9-5)-2$.
- When an operand like 5 has operators to its left and right, conventions are needed for deciding which operator applies to that operand.
- We say that the operator $+$ associates to the left, because an operand with plus signs on both sides of it belongs to the operator to its left.
- In most programming languages the four arithmetic operators, addition, subtraction, multiplication, and division are left-associative.

Associativity of Operators

- Some common operators such as **exponentiation** are **right-associative**.
- As another example, the **assignment operator =** in C and its descendants is **right associative**;
- that is, the expression **a=b=c** is treated in the same way as the expression **a=(b=c)**.
- Strings like **a=b=c** with a **right-associative operator** are generated by the following grammar:

$$\begin{array}{ll} \textit{right} & \rightarrow \textit{letter} = \textit{right} \mid \textit{letter} \\ \textit{letter} & \rightarrow \textit{a} \mid \textit{b} \mid \cdots \mid \textit{z} \end{array}$$

Associativity of Operators

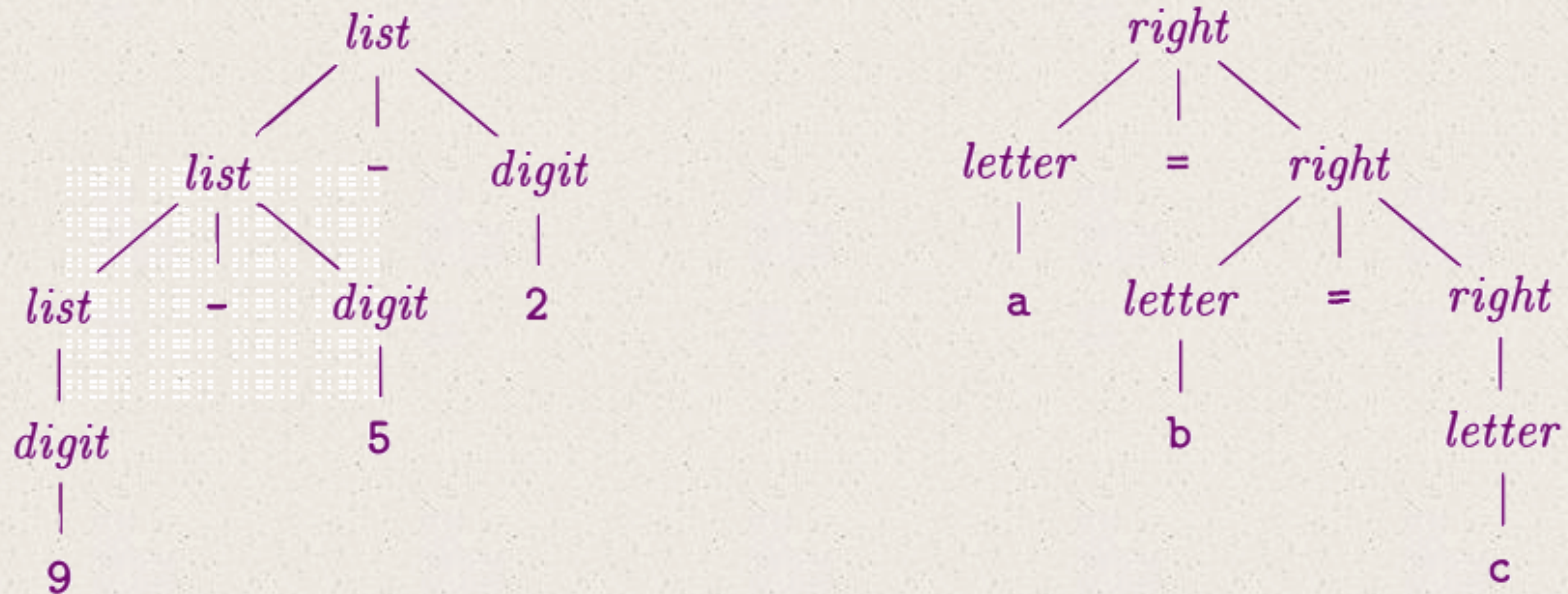


Figure 2.7: Parse trees for left- and right-associative grammars

Associativity of Operators

- The contrast between a parse tree for a **left-associative operator like -** and a parse tree for a **right-associative operator like =** is shown by **Fig. 2.7**.
- Note that the parse tree for **9-5-2** grows down **towards the left**,
- whereas the parse tree for **a=b=c** grows down **towards the right**.

Precedence of Operators

- Consider the expression $9+5*2$.
- There are two possible interpretations of this expression:
 - $(9+5)*2$ or $9+(5*2)$.
- The associativity rules for $+$ and $*$ apply to occurrences of the same operator, so they do not resolve this ambiguity.
- Rules defining the relative precedence of operators are needed when more than one kind of operator is present.

Precedence of Operators

- In ordinary arithmetic, multiplication and division have higher precedence than addition and subtraction.
- Therefore, 5 is taken by * in both $9+5*2$ and $9*5+2$;
- i.e., the expressions are equivalent to $9+(5*2)$ and $(9*5)+2$, respectively.

Precedence of Operators: Example 2.6

- A grammar for arithmetic expressions can be constructed from a table showing the associativity and precedence of operators.
- We start with the four common arithmetic operators and a precedence table, showing the operators in order of increasing precedence.
- Operators on the same line have the same associativity and precedence:

left-associative:	+	-
left-associative:	*	/

Precedence of Operators: Example 2.6

- We create two nonterminals *expr* and *term* for the *two levels of precedence*,
- and an extra nonterminal *factor* for generating *basic units* in expressions.
- The basic units in expressions are presently *digits* and *parenthesized expressions*.

$$factor \rightarrow \mathbf{digit} \mid (expr)$$

Precedence of Operators: Example 2.6

- Now consider the binary operators, $*$ and $/$, that have the **highest precedence**.
- Since these operators **associate to the left**, the productions are similar to those for **lists that associate to the left**.

$$\begin{array}{lcl} \textit{term} & \rightarrow & \textit{term} * \textit{factor} \\ & | & \textit{term} / \textit{factor} \\ & | & \textit{factor} \end{array}$$

Precedence of Operators: Example 2.6

- Similarly, *expr* generates lists of *terms* separated by the additive operators.

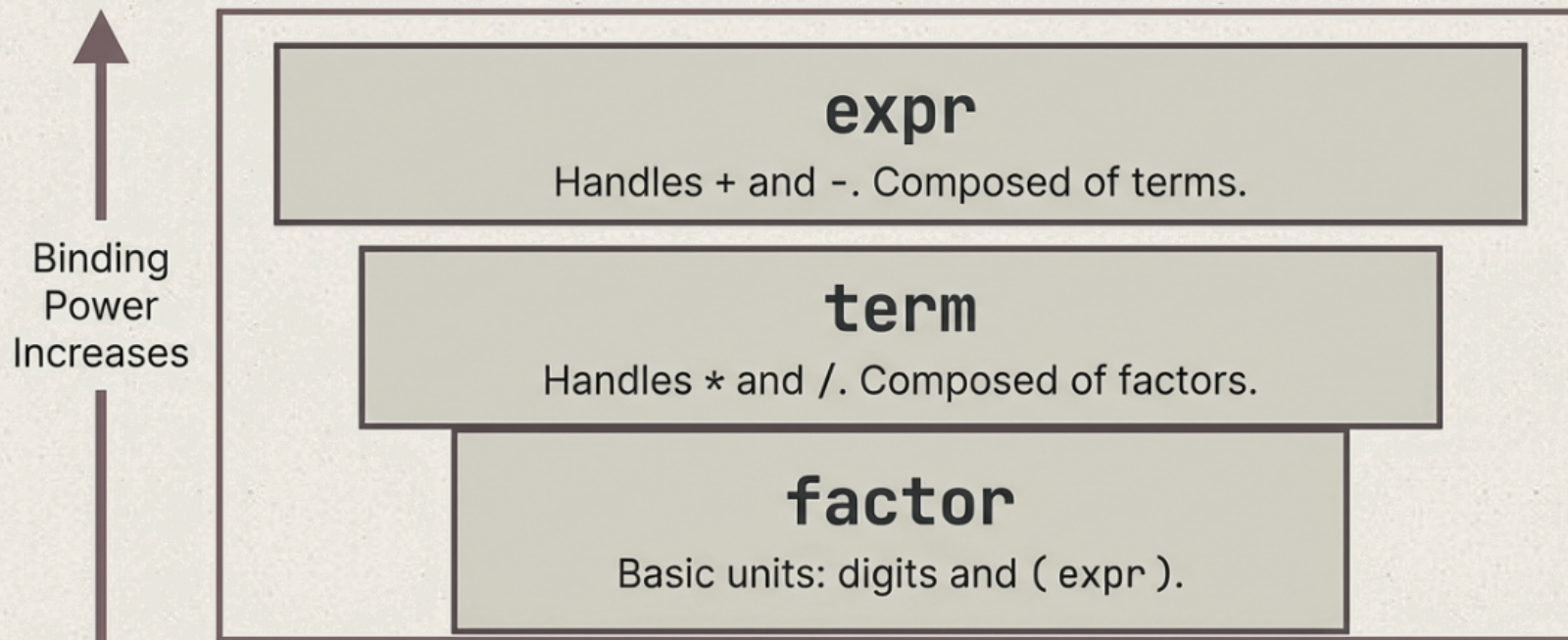
$$\begin{array}{lcl} \textit{expr} & \rightarrow & \textit{expr} + \textit{term} \\ & | & \textit{expr} - \textit{term} \\ & | & \textit{term} \end{array}$$

- The resulting grammar is therefore

$$\begin{array}{lcl} \textit{expr} & \rightarrow & \textit{expr} + \textit{term} \mid \textit{expr} - \textit{term} \mid \textit{term} \\ \textit{term} & \rightarrow & \textit{term} * \textit{factor} \mid \textit{term} / \textit{factor} \mid \textit{factor} \\ \textit{factor} & \rightarrow & \textbf{digit} \mid (\textit{expr}) \end{array}$$

Precedence of Operators: Example 2.6

Stratifying the Grammar



Example 2.7

- The statements defined by the (ambiguous) grammar in Fig. 2.8 are legal in Java.

```
stmt  →  id = expression ;  
        |  if ( expression ) stmt  
        |  if ( expression ) stmt else stmt  
        |  while ( expression ) stmt  
        |  do stmt while ( expression ) ;  
        |  { stmts }  
  
stmts →  stmts stmt  
        |  ε
```

Figure 2.8: A grammar for a subset of Java statements

Example 2.7

- In the **first** production for **stmt**, the terminal **id** represents any **identifier**.
- The productions for **expression** are **not shown**.
- **Java** allows **a=b=c**, which this grammar **does not**.
- The nonterminal **stmts** generates a possibly **empty list of statements**.
- The **second** production for **stmts** generates the empty list **ε**.
- The **first** production generates a **possibly empty list of statements followed by a statement**.
- The placement of **semicolons** is subtle; **they appear at the end of every body that does not end in stmt**.

Exercises

- Page 74.
 - Exercise 2.2.1 -> Exercise 2.2.5.